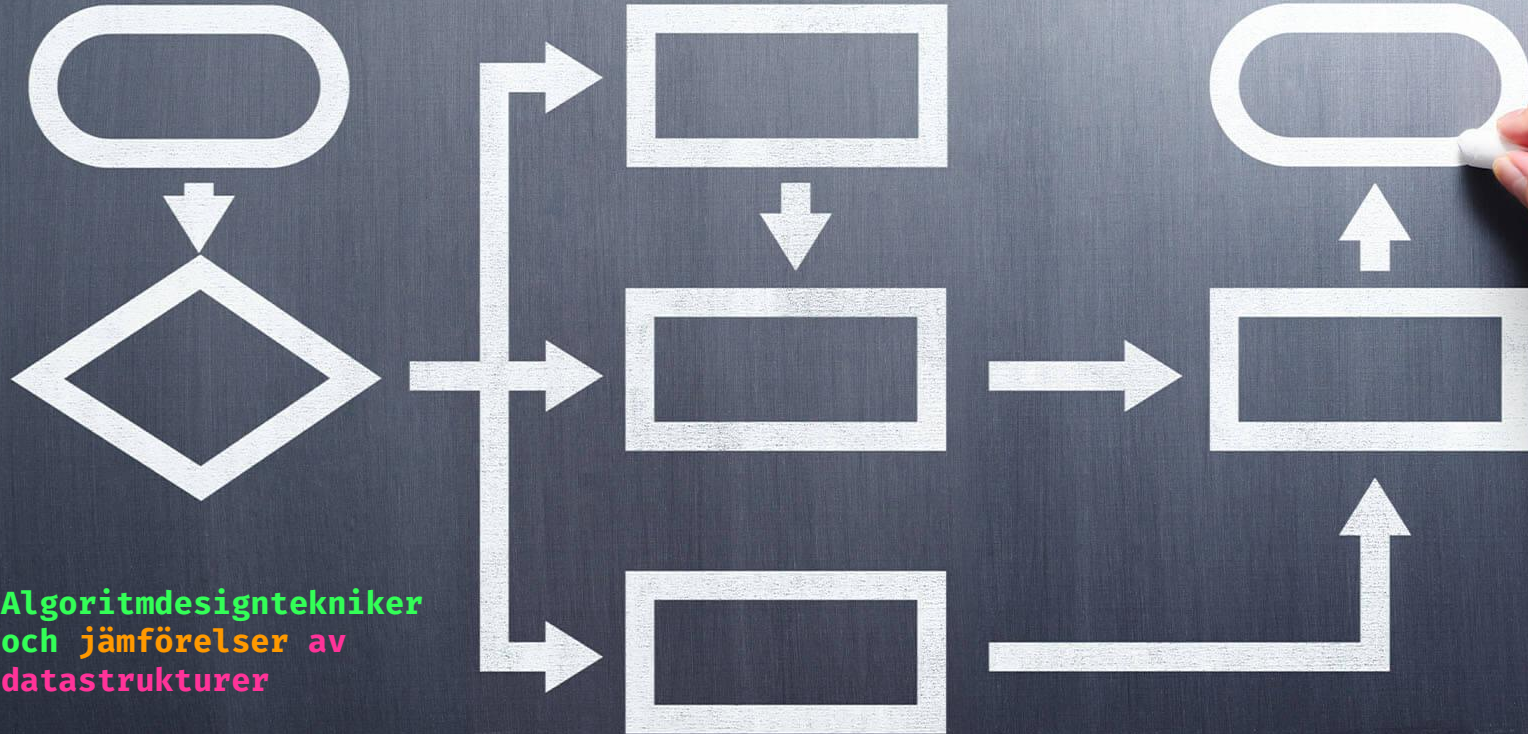


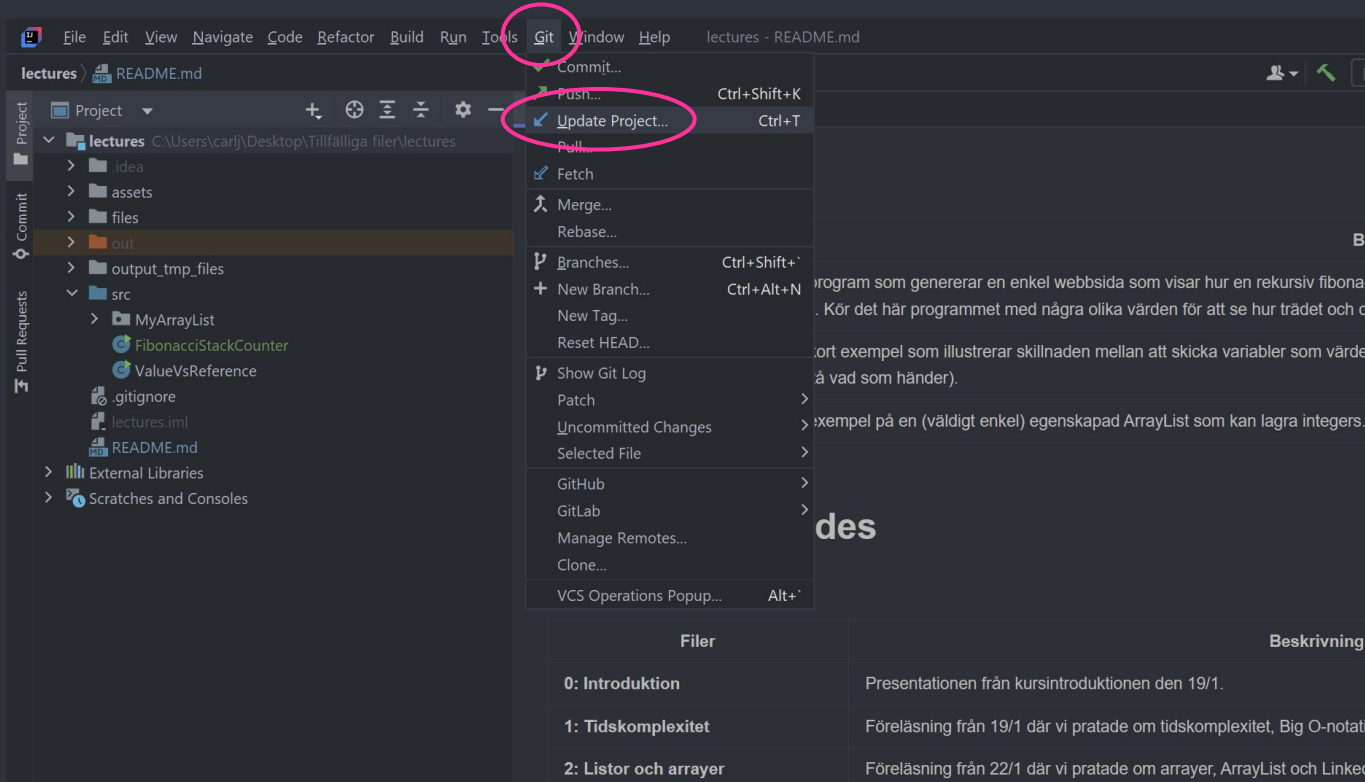
Algoritmer och Datastrukturer

Föreläsning 9



Algoritmdesigntechniker
och jämförelser av
datastrukturer

<https://github.com/carljohanj/algo>



The screenshot shows the IntelliJ IDEA interface with the 'Git' menu open. The 'Update Project...' option is highlighted with a red circle. The menu also includes options like 'Commit...', 'Push...', 'Pull...', 'Fetch', 'Merge...', 'Rebase...', 'Branches...', 'New Branch...', 'New Tag...', 'Reset HEAD...', 'Show Git Log', 'Patch', 'Uncommitted Changes', 'Selected File', 'GitHub', 'GitLab', 'Manage Remotes...', and 'Clone...'. The 'VCS Operations Popup...' option is also visible at the bottom of the menu.

The background shows the project structure in the 'Project' tool window, with the 'lectures' directory selected. The 'src' directory contains files like 'MyArrayList', 'FibonacciStackCounter', 'ValueVsReference', '.gitignore', 'lectures.iml', and 'README.md'. The 'out' directory is also visible.

On the right side of the screenshot, there is a table with the following content:

Filer	Beskrivning
0: Introduktion	Presentationen från kursintroduktionen den 19/1.
1: Tidskomplexitet	Föreläsning från 19/1 där vi pratade om tidskomplexitet, Big O-notati
2: Listor och arrayer	Föreläsning från 22/1 där vi pratade om arrayer, ArrayList och Linke

Sista veckorna på kursen

- Vi har inte så många föreläsningstillfällen kvar på kursen: schemat är en röra pga schemaläggningen för kursen fick göras om i all hast, men vi har egentligen bara tre tillfällen kvar inklusive i dag
- Nästa är på torsdag när vi påbörjar projektet. Jag kommer köra en projektintroduktion på sal, och rekommenderar att alla som vill göra projektet är på plats då
- Ytterligare ett tillfälle nästa vecka på måndag, när vi har det mysigt och kör en heldag med Test-Driven Design (hurra!)
- Inga fler tillfällen planerade efter det, men jag kör gärna ett repetitionstillfälle om det finns intresse; vi kan t.ex. repetera sånt ni tycker varit svårt att greppa och gå genom några gamla tentor tillsammans
- Finns det ett intresse för ett sådant tillfälle?

Sista veckorna på kursen

Här är vi i dag! \o/

	Tid	Kurs	Program	Grupp	Lokal	Moment
v 8	Mån 2026-02-16					
	10:15 - 12:00	Algoritmer och datastrukturer			B23, Campus Gotland	Föreläsning
	13:15 - 15:00	Algoritmer och datastrukturer			E31, Campus Gotland	Föreläsning
	Tors 2026-02-19					
	10:15 - 12:00	Algoritmer och datastrukturer			B23, Campus Gotland	Föreläsning
	13:15 - 15:00	Algoritmer och datastrukturer			D24, Campus Gotland	Laboration
v 9	Mån 2026-02-23					
	10:15 - 12:00	Algoritmer och datastrukturer			B25, Campus Gotland	Föreläsning
	13:15 - 15:00	Algoritmer och datastrukturer			B22, Campus Gotland	Föreläsning
v 10	Mån 2026-03-02					
	10:15 - 12:00	Algoritmer och datastrukturer			D22, Campus Gotland	Föreläsning
	13:15 - 15:00	Algoritmer och datastrukturer			E30, Campus Gotland	Laboration
v 11	Mån 2026-03-09					
	10:15 - 12:00	Algoritmer och datastrukturer			D25, Campus Gotland	Seminarium
	13:15 - 15:00	Algoritmer och datastrukturer			B22, Campus Gotland	Laboration
	Tors 2026-03-12					
	10:15 - 12:00	Algoritmer och datastrukturer			B14, Campus Gotland	Seminarium
	13:15 - 15:00	Algoritmer och datastrukturer			B15, Campus Gotland	Seminarium

Onsdag 11 mars:
Seminarium för
projektuppgift

Projektstart och projektgenomgång

Test-Driven Design (heldag)

Torsdag 12/3: tentarepetition?
Alt. Torsdag 5/3 (dagen innan projektdeadline)?

Labbtillfällan

- Tre stycken kvar på kursen:
 - * Torsdag 19/2 (nu på torsdag)
 - * Måndag 2/3 (inte nästa måndag, utan den efter det)
 - * Måndag 9/3
- Känn ingen panik: det är fortfarande gott om tid
- Tänk dock på att projektet kommer ta upp mycket av er tid under de kommande veckorna, och det kan vara bra att beta av så många av labbarna som möjligt innan det startar

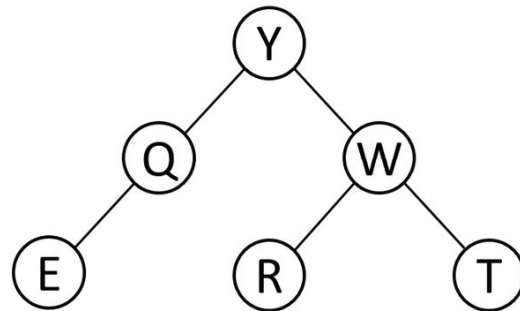
Lite repetition!

Trädtraversering

2. Utifrån trädet och koden nedan:

1. I vilken ordning kommer noderna besökas om vi skickar in noden Y till metoden?
2. I vilken ordning kommer noderna besökas om vi skickar in noden W till metoden?
3. Vilken kallas traverseringen som koden medför?

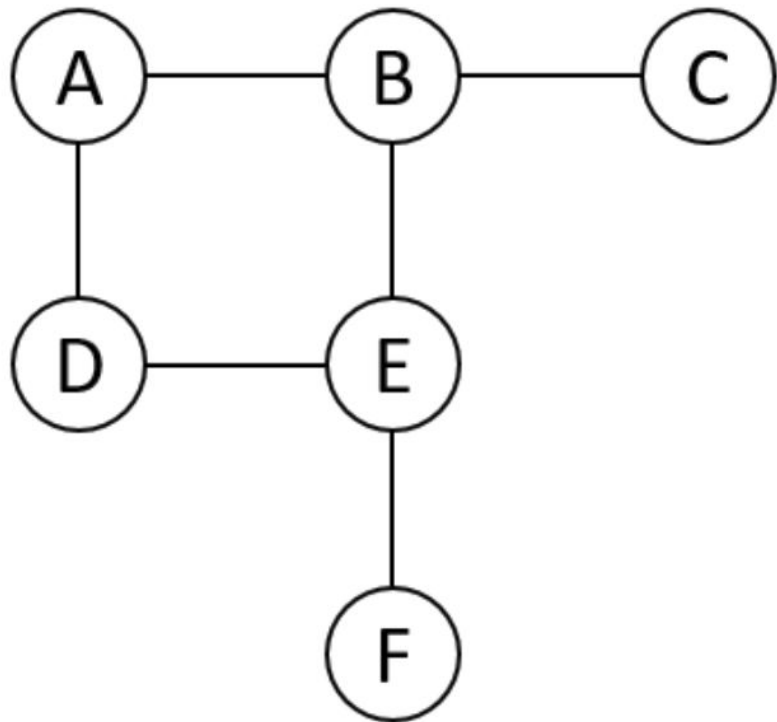
```
public void visit(TreeNode node) {  
    if(node != null) {  
        visit(node.getLeftChild());  
        visit(node.getRightChild());  
        System.out.println(node.getName());  
    }  
}
```



4. Vad blir tidskomplexiteten i fråga 2.2 (start vid W)?

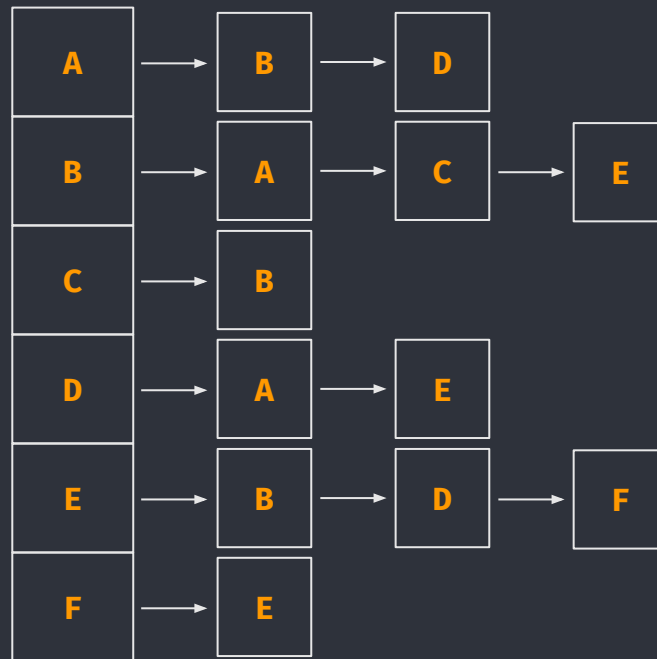
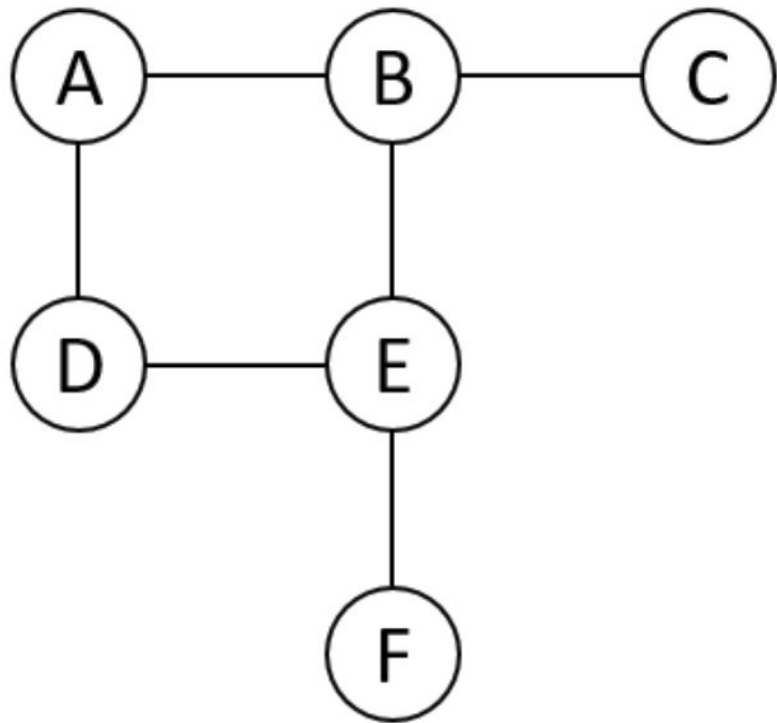
Svar: $O(k)$ eftersom vi bara följer referenser vid traversering, och varje nod pekar mot nästa nod; alltså **inte** $O(k \cdot \log n)$

Rita grannmatris för graf



	A	B	C	D	E	F
A	0	1	0	1	0	0
B	1	0	1	0	1	0
C	0	1	0	0	0	0
D	1	0	0	0	1	0
E	0	1	0	1	0	1
F	0	0	0	0	1	0

Rita grannlista för graf



Grafsökning

6. Om vi ska implementera en iterativ djupet-först-traversering av en graf, vilken datastruktur bör vi använda för att lagra noderna?
7. Om vi ska implementera en bredden-först-traversering av en graf, vilken datastruktur bör vi använda för att lagra noderna?

- 6) En stack, eftersom en djupet först-sökning behöver kunna backtracka och utforska alternativa vägar. Detta beteende imiterar en stack perfekt, där vi lägger på och tar av saker (tänk på noderna som metoodanrop).
- 7) En kö, eftersom vi behöver processera noderna i den ordning de läggs in. Vi börjar alltid att ta från början av en kö och kollar vilka kanter som finns för varje nod vi besöker.

Jämförelse av datastrukturer

	Lista Samtliga	Lägga till	Ta bort	Söka	Intervall
Array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(k)$
ArrayList	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(k)$
LinkedList	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$(n+k)$
TreeMap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
HashMap	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(k)$

Array

Fördelar:

- Snabb random access ($O(1)$ pga indexering)
- Simpel
- Minneseffektiv (ingen extra overhead)
- Cachevänlig (sekventiellt minne)

Nackdelar:

- Fixerad storlek: måste skapa ny array om vi vill förstora
- Insättning/borttagning är dyra ($O(n)$)
- Måste skifta alla elementen manuellt

Bra när:

- Storlek är känd på förhand och inte behöver utökas
- Man behöver snabb indexbaserad åtkomst
- Prestanda och minne spelar stor roll

Array: operationer

Lista samtliga: $O(n)$

```
public void listAll()
{
    for(int i : array)                //O(n) *
    {
        System.out.print(i + " ");    //O(1)
    }
}
```

Lägga till element: $O(n)$

```
public void addElement(int i)
{
    int[] newArray = new int[array.length + 1]; //O(n) +

    for(int i = 0; i < array.length; i++)      //O(n) *
    {
        newArray[i] = array[i];                //O(1) +
    }

    newArray[newArray.length - 1] = i;          //O(1) +
    array = newArray;                          //O(1)
}
```

Array: operationer

Ta bort element: $O(n)$

```
public void removeElement(int index)
{
    int[] newArray = new int[array.length - 1]; //O(n) +
    int j = 0; //O(1) +

    for (int i = 0; i < array.length; i++) //O(n) *
    {
        if (i != index)
        {
            newArray[j++] = array[i]; //O(1) +
        }
    }
    array = newArray; //O(1)
}
```

Array: operationer

Söka element: $O(n)$

```
public int find(int find)
{
    for (int i = 0; i < array.length; i++)    //O(n) *
    {
        if (array[i] == find)                //O(1) +
        {
            return i;                        //O(1) +
        }
    }
    return -1;                               //O(1)
}
```

Hämta intervall: $O(k)$

```
public int[] subarray(int start, int end)
{
    int[] subarray = new int[end - start + 1];    //O(k) +

    for (int j = 0, int i = start; i <= end; i++) //O(k) *
    {
        subarray[j++] = array[i];                //O(1) +
    }
    return subarray;                             //O(1)
}
```

ArrayList

Fördelar:

- Snabb random access ($O(1)$ precis som array)
- Snabb insättning i slutet ($O(1)$ om den inte behöver förstora sig själv)
- Dynamisk: förstoras automatiskt

Nackdelar:

- Insättning/borttagning i mitten är $O(n)$
- Förstoring är $O(n)$ både tidskomplexitet och platskomplexitet när det inträffar
- Dålig för program där vi ofta vill lägga till eller ta bort i mitten av listan

Bra när:

- Vi främst behöver läsa från en datastruktur
- Vi främst behöver lägga till värden på slutet
- Vi ofta behöver indexbaserad åtkomst

ArrayList: operationer

Lista samtliga: $O(n)$

```
public void listAll()
{
    for (int i : arrayList)                //O(n) *
    {
        System.out.print(i + " ");        //O(1)
    }
}
```

Lägga till element: $O(1)$

```
public void addElement(int e)
{
    arrayList.add(e);                      //O(1) i normalfallet, men O(n) om den
                                           //underliggande arrayen behöver förstoras
}
```

Lägga till element på valfri plats: $O(n)$

```
public void addElement(int index, int e)
{
    arrayList.add(index, e);              //O(n)
}
```

ArrayList: operationer

Ta bort element: $O(n)$

```
public void removeElement(int index)
{
    arrayList.remove(index);           //O(n)
}
```

Söka element: $O(n)$

Hämta intervall: $O(k)$

LinkedList

Fördelar:

- Snabb insättning i båda ändarna ($O(1)$ på dubbellänkad lista)
- Behöver aldrig förstöras eller initialiseras till viss storlek eftersom den är spridd i minnet
- Behöver inte sekventiell minnesallokering

Nackdelar:

- Långsam random access ($O(n)$ eftersom vi måste loopa genom alla noder tills vi hittar rätt plats)
- Mer overhead minnesmässigt
- Dålig cacheeffektivitet

Bra när:

- Vi ofta behöver lägga till i början eller slutet
- Vi behöver lägga till i mitten och redan vet var noden är

LinkedList: operationer

Lista samtliga: $O(n)$

```
public void listAll()
{
    for (int i : linkedList)           //O(n)
    {
        System.out.print(i + " ");    //O(1)
    }
}
```

Lägga till element: $O(1)$

```
public void addElement(int e)
{
    linkedList.add(e);                 //O(1)
}
```

Lägga till element på valfri plats: $O(n)$

```
public void addElement(int index, int e)
{
    linkedList.add(index, e);          //O(n)
}
```

Ta bort element: $O(1)$

```
public void removeElement(int index)
{
    linkedList.remove(index);           //O(1) i bästa fall (första eller sista
                                        //elementet)
}
```

LinkedList: operationer

Söka element: $O(n)$

```
public int find(int find)
{
    int i = 0; //O(1) +

    for (Iterator<Integer> itr = linkedList.iterator(); itr.hasNext(); i++) //O(n) *
    {
        if (itr.next() == find) { return i; } //O(1) +
    }
    return -1; //O(1)
}
```

Hämta intervall: $O(n + k)$

```
public List<Integer> sublist(int start, int end)
{
    List<Integer> sublist = new LinkedList<>(); //O(1) +
    int i = start; //O(1) +

    for (Iterator<Integer> itr = linkedList.listIterator(start); itr.hasNext() && i <= end; i++) //O(n + k) *
    {
        sublist.add(itr.next()); //O(1) +
    }
    return sublist; //O(1)
}
```

TreeMap

Fördelar:

- Alltid sorterade nyckel-värde-par vid insättning
- $O(\log n)$ för lägga till/ta bort/söka
- SubMap returnerar en vy på konstant tid ($O(1)$)

Nackdelar:

- Lite långsammare än HashMap för simpel lookup
- Mer overhead minnesmässigt

Bra när:

- Du behöver sorterade nycklar
- Du ofta behöver hämta ut spann av data (blir aldrig sämre än $O(k)$ tack vare subMap)
- Du behöver iterera över samlingar ofta

TreeMap: operationer

Lista samtliga: $O(n)$

```
public void listAll()
{
    for (Map.Entry<String, Integer> e : treeMap.entrySet()) //O(n)
    {
        System.out.println(e.getKey() + " " + e.getValue()); //O(1)
    }
}
```

Lägga till element: $O(\log n)$

```
public void addElement(int key, int e) { treeMap.put(key, e); } //O(log n)
```

Ta bort element: $O(\log n)$

```
public void removeElement(int key) { treeMap.remove(key); } //O(log n)
```

Söka element: $O(\log n)$

```
public int find(int find) { return treeMap.get(find); } //O(log n)
```

Hämta intervall: $O(1)$

```
public Map<Integer, Integer> subMap(int start, int end)
{
    return treeMap.subMap(start, end); //O(1)
}
```

HashMap

Fördelar:

- Snabb lookup ($O(1)$ i genomsnitt)
- Snabb insättning/borttagning ($O(1)$ i genomsnitt)
- Bra för nyckelbaserad access

Nackdelar:

- Ingen inbördes ordning bevaras, varken insättning eller sortering
- Sämsta fallet kan degradera till $O(n)$
- Ingen effektiv uthämtning för spann
- Hashsekvensen kan skapa stora konstanter

Bra när:

- Snabb lookup är hög prioritet
- Ordning inte spelar någon roll
- Vi allmänt behöver lagra key-value-par (används ofta som default map i applikationer)

HashMap: operationer

Lista samtliga: $O(n)$

```
public void listAll()
{
    for (Map.Entry<String, Integer> e : hashMap.entrySet()) //O(n)
    {
        System.out.println(e.getKey() + " " + e.getValue()); //O(1)
    }
}
```

Lägga till element: $O(1)$

```
public void addElement(int key, int e) { hashMap.put(key, e); } //O(1) i normalfallet men  $O(\log n)$  i värsta fall
```

Ta bort element: $O(1)$

```
public void removeElement(int key) { hashMap.remove(key); } //O(1) i normalfallet men  $O(\log n)$  i värsta fall
```

Söka element: $O(1)$

```
public int find(int find) { hashMap.get(find); } //O(1) i normalfallet men  $O(\log n)$  i värsta fall
```

Hämta intervall: $O(k)$ i normalfallet men $O(k \log n)$ i värsta fall

```
public Map<Integer, Integer> subMap(int start, int end)
{
    Map<Integer, Integer> submap = new HashMap<>(end - start + 1); //O(k)
    for (int i = start; i <= end; i++) //O(k)
    {
        submap.put(i, hashMap.get(i)); //O(1) i normalfallet men  $O(\log n)$  i värsta fall
    }
    return submap; //O(1)
}
```

kaffepaus(15);



Algoritmdesigntechniker

Brute force, Greedy, Divide-and-conquer,
Dynamisk Programming, Backtracking

Jämförelse, olika sorteringsalgoritmer

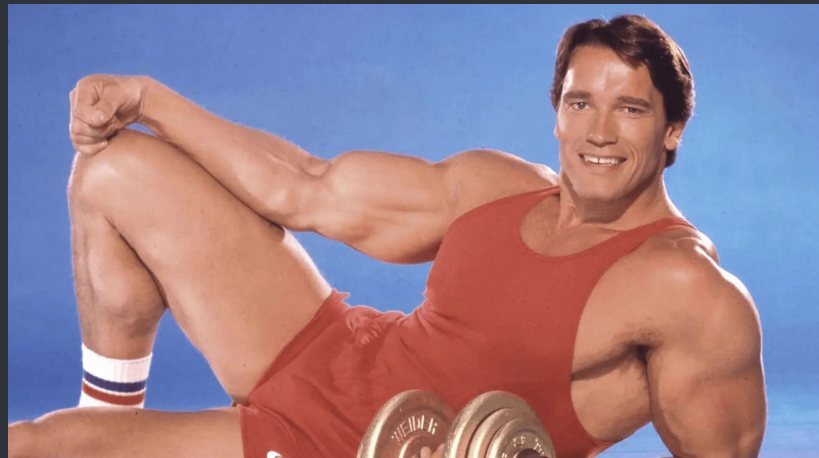
Algoritm	Tidskomplexitet (Bäst/Sämst)	In-Place	Stabil	Används
QuickSort	$O(n \cdot \log n)$ / $O(n^2)$	Ja	Nej	Stora dataset som behöver in-place-sortering
MergeSort	$O(n \cdot \log n)$	Nej	Ja	När stabilitet är viktigt
InsertionSort	$O(n)$ / $O(n^2)$	Ja	Ja	Små eller mestadels sorterade samlingar
BubbleSort	$O(n^2)$	Ja	Ja	Helst inte, men ok för små dataset
SelectionSort	$O(n^2)$	Ja	Nej	Används sällan men föredras över BubbleSort pga färre swaps

- Giltiga algoritmer
- Används främst i lärosyfte

Brute force

- Ett utmärkt första steg är ofta att **skapa någonting som fungerar innan vi börjar optimera det**
- Brute force är en approach som **testar alla möjliga kombinationer tills den hittar rätt**
- “Brute force-attacker” på inloggningssidor handlar ofta om att en hackare provar alla möjliga lösenord tills hen hittar en kombination som är rätt
- **Det här är varför man är orolig för** att kryptering ska gå att knäcka i framtiden med **kvantdatorer**: om man får dem att fungera skulle de kunna brute force:a kryptering som skulle ta en vanlig dator miljarder av år
- Brute force kan ibland vara det enda sättet att göra någonting på, men oftast finns det bättre alternativ

Exempel: **Linjär sökning**, **Bubble Sort**



Tvåsummaproblemet

- Givet ett set med nummer, hur kan vi kolla om två av talen adderade med varandra resulterar i ett visst värde?

Exempel:

Input: [2, 11, 15, 7]

Värde: 9

Output: [0, 3] //Eftersom värdena på plats 0 och 3 är de vi letar efter:
2+7 är ju 9

- Brute force-lösning: kolla varje par
- **$O(n^2)$ tid eftersom vi använder två for-loopar:** vi kollar varje värde mot alla andra värde och ser vad additionen av dem blir

Tvåsummaproblemet

Brute force-lösning:

Tvåsummaproblemet

Optimerad lösning med HashMap: I stället för att fråga: "Är `nums[i] + nums[j]` lika med `target`?" frågar vi: "Har jag redan sett numret som skulle matcha `target` om jag adderade det med det här numret?"

```
public int[] returnPair(int[] nums, int target)
{
    HashMap<Integer, Integer> map = new HashMap<>();    //O(1)

    for (int i = 0; i < nums.length; i++)                //O(n) *
    {
        int needed = target - nums[i];                    //O(1) +
                                                         //O(n)!
        if (map.containsKey(needed))                     //O(1) +
        {
            return new int[]{ map.get(needed), i };       //O(1) +
        }

        map.put(nums[i], i);                             //O(1) +
    }

    return new int[]{ 0, 0 };                             //O(1)
}
```


Greedy

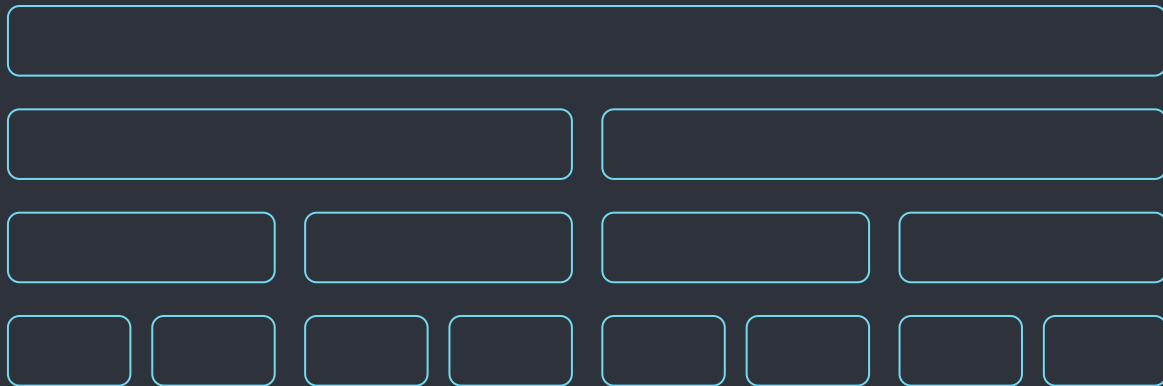
- Algoritmer som är giriga **arbetar i steg**: i varje steg väljer de det som är bäst just nu. De väljer alltså det **lokalt bästa alternativet** och antar att det leder till det globalt bästa
- När den väl tagit ett beslut håller den alltid fast vid det och ändrar sig aldrig
- Dijkstras Algoritm plottar den kortaste vägen genom en viktad graf och har revolutionerat path finding i nätverk och system, men fungerar bara så länge kantvikten inte är negativ
- Negativ kantvikt kan t.ex. representera en vinst i monetära system, rabatter på varuköp, bonuspoäng i spel, osv - allt som representerar en vinst ist. för en kostnad

Exempel: Selection Sort, Dijkstras Algoritm



Divide-and-conquer

- Algoritmer som använder sig av divide-and-conquer delar upp problem i två eller fler delar som kan lösas på samma sätt som huvudproblemet
- Väldigt väl lämpade för rekursiva implementationer
- Många av de mest kraftfulla algoritmerna och datastrukturerna för sökning, sortering och lagring av objekt använder sig av den här tekniken



Exempel: Binär Sökning, Merge Sort, Quick Sort, Binära Träd

Dynamisk programmering

- Optimeringsteknik som går ut på att spara redan gjorda beräkningar och återanvända dem vid behov, i stället för att utföra samma beräkning om och om igen
- Fungerar för problem som har två egenskaper:
 - * De kan brytas ner i mindre delproblem
 - * Dessa delproblem upprepas väldigt många gånger
- Dynamisk programmering fungerar INTE för t.ex. Merge Sort eftersom den delar in datan i självständiga mindre arrayer som den sedan sorterar: inget arbete upprepas

Exempel: Fibonacci (optimerad), Matrimultiplikation, Lägsta gemensamma delsekvens

Dynamisk programmering

- Dumt namn, har ingenting att göra med datorprogrammering: begreppet är matematiskt och myntades redan på 1940-talet när man satt och crunchade tal för hand på papper
- Det finns två typer av dynamisk programmering:

Memoisering (Memoization)

Top-down-approach
Rekursiv approach

Tabulering (Tabulation)

Bottom-up-approach
Iterativ approach

- Idén är att **spara resultaten från delproblem** så att man slipper göra samma kalkyleringar flera gånger

Backtracking

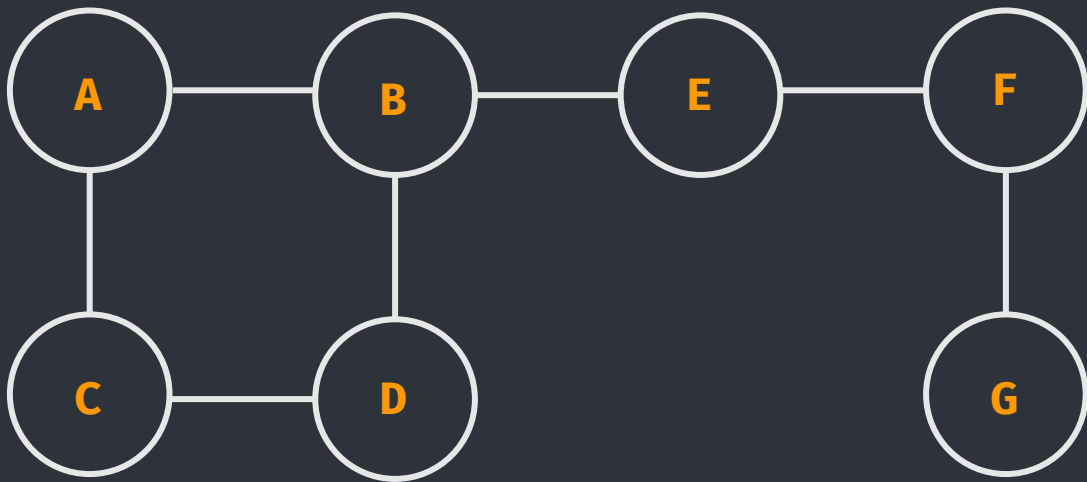
- Algoritmer som backtrackar söker systematiskt efter lösningar bland alla alternativ. Det här skiljer dem från giriga algoritmer som aldrig ändrar sig när de tagit ett beslut: om en backtrackingalgoritm hamnar i en återvändsgränd går den tillbaka till föregående val och provar ett nytt alternativ
- Backtracking påminner en del om brute force, men till skillnad från brute force återvänder den tidigt om en väg inte funkar
- Ta ett vägvalsträd som exempel: brute force testar varenda nod hela vägen ner. Om en väg blir ogiltig halvvägs ner kommer en backtrackingalgoritm däremot att genast återvända



“Nähä, den här vägen funkar inte heller...”

Exempel: Djupet först-sökning i grafer, Knight's Tour

Backtracking: djupet först i graf



DFS:

Djupet Först-Sökning

Använd en stack

Besöksordning (utgår från noden A): A, B, D, C, E, F, G

Backtracking: Knight's Tour

- En riddare (hästpjäsen) i schack kan röra sig i en L-formation
- Om den startar på en viss ruta på ett schackbräde (8×8 rutor), kan den röra sig på ett vis som gör att den besöker varje ruta på brädet exakt en gång?
- Tidskomplexiteten för en naiv lösning är $O(8^N)$!
- Skulle ta miljarder år att brute force:a
- Genom att optimera backtrackingalgoritmer kan man dock få ner den till polynomisk tid



Lunch(60);

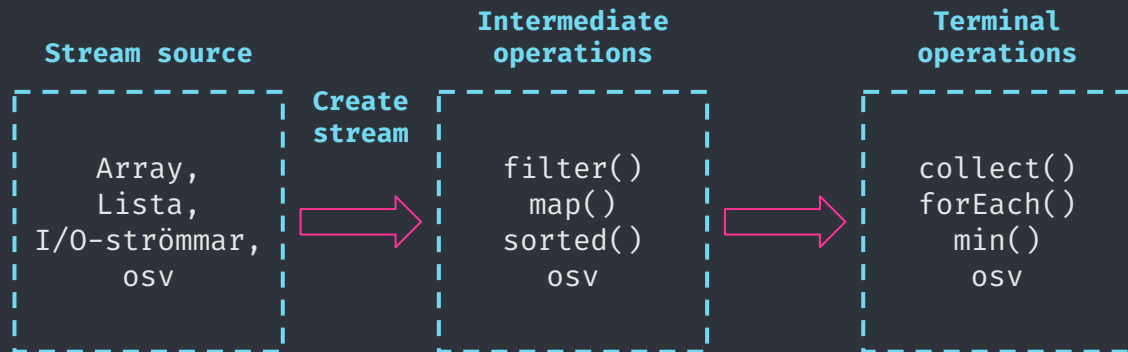
Vi ses igen i E31 13.15

Streams och lambdas

Moderna språkfeatures

Java Streams

- Streams är **inte en datastruktur**, och de modifierar inte heller den datastruktur som de används på
- Ska inte förväxlas med IO-strömmar som är något helt annat
- **Streams ersätter kod som normalt skulle använt loopar** för att beräkna någonting
- Ger ofta tydlig och kompakt kod, men gör inte koden mer effektiv (snabbare) normalt sett utan är bara syntaktiskt socker

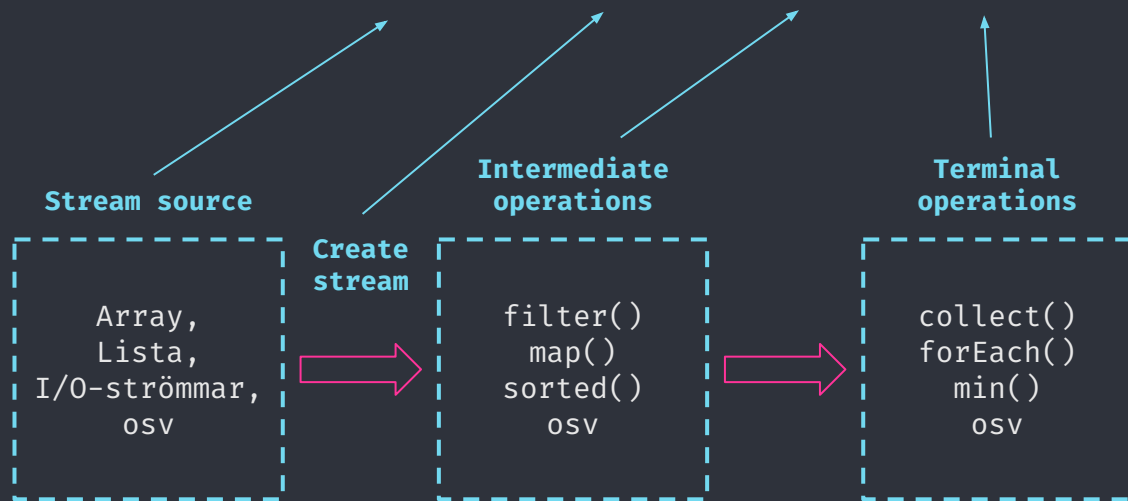


Java Streams: exempel

```
List<Integer> integerList = Arrays.asList(1, 2, 1, 3, 2, 5);
```

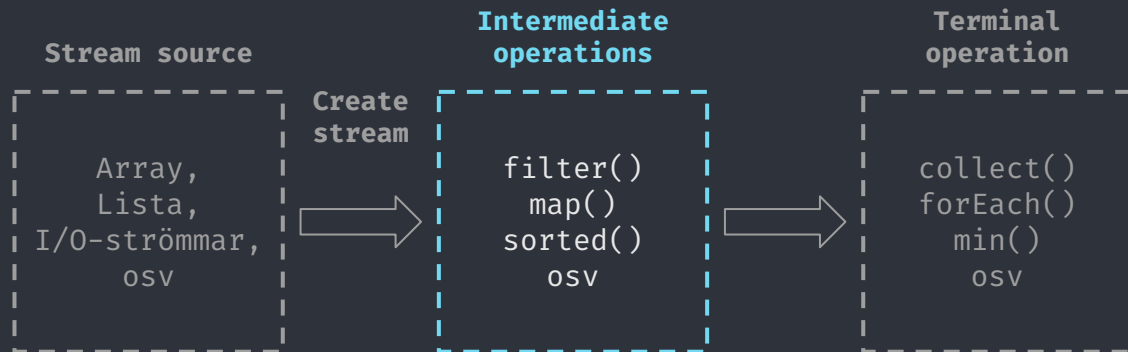
Filtrerar ut unika tal (1, 2, 3, 5) och räknar hur många de är (4):

```
int distinctNumbers = integerList.stream().distinct().count();
```



Intermediate operations

- Två typer av operationer: intermediate och terminal
- **Intermediate är en kedja av metodanrop** där en eller flera saker utförs på rad
- De förvandlar en stream till en annan stream
- Kan ses **som en mängd filter för datan**: outputen av ett filter blir inputen för nästa filter



Intermediate operations: Exempel

- **map()**: Omvandlar varje element till någonting annat

```
List<String> upper =  
names.stream().map(String::toUpperCase).collect(Collectors.toList());
```

- **filter()**: Behåller bara element som matchar ett visst villkor

```
List<Integer> evens =  
numbers.stream().filter(n → n % 2 == 0).collect(Collectors.toList());
```

- **distinct()**: Filtreerar bort kopior

```
List<Integer> unique = numbers.stream().distinct().collect(Collectors.toList());
```

- **sorted()**: Sorterar element med en Comparator, som vi gått genom tidigare

```
.sorted(Comparator.comparing(Person::getAge))
```

- **limit(n)**: Behåller bara de första n elementen

```
numbers.stream().limit(5).forEach(System.out::println);
```

Terminal operations

- Terminal är den **avslutande operationen**
- Dessa kedjas inte till skillnad från intermediate operations: **det finns bara en avslutande operation** per stream
- Den processerar ingenting mer utan returnerar bara ett resultat
- `forEach()` itererar t.ex. genom resultatet men beräknar inte någonting nytt



Intermediate operations: Exempel

- **collect()**: Samlar element i en container, t.ex. en lista:

```
List<String> upper =  
names.stream().map(String::toUpperCase).collect(Collectors.toList());
```

- **forEach()**: Utför en operation för varje element i streamen:

```
numbers.stream().limit(5).forEach(System.out::println);
```

- **count()**: Returnerar antalet element:

```
long count = names.stream().filter(s → s.length() > 3).count();
```

- **findFirst()** / **findAny()**: Returnerar en Optional (kanske finns, kanske inte!)

```
Optional<String> first = names.stream().filter(s → s.startsWith("A")).findFirst();
```

[Kodexempel]

Koden finns i paketet `StreamExample`

Repetition: Metodreferensoperatoren

- Någonting händer på den här raden i vår kod:

```
list.sort(Comparator.comparingInt(Person::getAge));
```



- Dessa dubbelkolon **kallas för metodreferensoperatoren**
- Vi använder den här operatoren när vi vill berätta för koden att den ska använda en viss metod – i det här fallet metoden `getAge()` i klassen `Person` – för att göra något. Vi kan inte skriva så här:

```
list.sort(Comparator.comparingInt(Person.getAge()));
```

- ...eftersom det i så fall **skulle anropa metoden `getAge()` direkt på den här raden**. Vi vill bara att `comparingInt()` ska använda den när den gör sina inre jämförelser! Vi **refererar därför till metoden** vi vill att den ska anropa när det är dags att göra så

Lambda: anonyma metoder

Låt oss diskutera den här raden:

```
filter((s) -> s.length() == 5)
```

- Vi är vana vid att skicka **data** till metoder: vi skickar in en int, eller en sträng, eller en samling med ints eller strängar, som argument när vi anropar en metod som den sedan kan göra någonting med

Exempel:

```
sortArray(int[] array) //int[] array är parametern som beskrivs av själva  
                        //metoden  
sortArray(minArray);   //minArray är argumentet vi anropar metoden med
```

- Men hur gör vi om vi skulle vilja skicka **ett beteende** i stället? Vi har ju lärt oss att beteende ("behavior" på engelska) är något som opererar på data - dvs metoder!
- **Faktum är att vi redan gjort just detta** när vi jobbade med **Comparator**! låt oss titta på metodreferensoperatören igen:

Lambda: anonyma metoder

- Metodreferensoperatören in action:

```
list.sort(Comparator.comparingInt(Person::getAge));
```

- Här säger vi till `comparingInt()`: "Använd metoden `getAge()` för att sortera `Person`-objekten i `list`!" Vi skickar alltså ett beteende som argument!
- I fallet ovan existerar redan `getAge()` inuti `Person`-klassen, och därför refererar vi bara till den så att `Comparator`n förstår var den ska hitta den någonstans
- Men **vi skulle också kunna skapa en helt ny metod och skicka den som argument** i stället. Och det är vid sådana här tillfällen som vi använder någonting som kallas för lambda
- En lambda är en anonym metod: det vill säga, den har inget namn och är inte definierad i någon fil på förhand
- **Det är ett sätt för oss att skicka en ny metod som argument i stället för ett värde**

Lambda: exempel

Parametrar ()
Men inget
metodnamn!

- En anonym metod har inget namn, utan bara parametrar, samt en speciell lambdaoperator:

`(s) -> s.length() == 5`

- Vi säger: "För ett element s, kolla om längden på s är 5"
- Det här är **exakt samma sak** som om vi skulle skriva:

```
public boolean hasLengthFive(String s)
{
    return s.length() == 5;
}
```

- `filter()` vet att s refererar till en sträng eftersom streamen vi skapat innehåller strängar: vi plockade ju ut namnen på alla personer med `map(Person::getName)`
- Filter kör sedan den här anonyma metoden för alla strängar i streamen

Streams med lambda

- Lambdas går att använda på alla ställen där en metod har ett så kallat funktionellt interface ("functional interface") som en av sina parametrar
- Ett funktionellt interface är ett interface som **innehåller exakt en metod**
- Vi kan skapa egna funktionella interface om vi vill, men det är när vi arbetar med saker som streams som de är verkligt kraftfulla
-

[Kodexempel]

Koden finns i paketet `LambdaExample`

På torsdag: projekt!

- På torsdag kommer vi att introducera projektuppgiften på kursen
- Ni kommer ha två veckor på er att göra det här projektet, och det har en hård deadline: missar man den får man vänta till maj eller augusti
- När ni lämnat in projektet kommer ni få någon annans projekt som ni sedan ska opponera på vid ett seminarium
- Vi kommer gå genom allting på torsdag förmiddag, så det är en bra idé att inte missa det föreläsningstillfället